

Report conversation

Given the following C++17 function that utilises Qt APIs:

```
void TTextEdit::expandSelectionToWords()
{
    int yind = mPA.y();
    int xind = mPA.x();
    for (; xind >= 0; --xind) {
        if (mpBuffer->lineBuffer.at(yind).at(xind) == ' ' || mpHost->mDoubleClickIgnore.c
            break;
        }
    }
    mDragStart.setX(xind + 1);
    mPA.setX(xind + 1);

    yind = mPB.y();
    xind = mPB.x();
    for (; xind < static_cast<int>(mpBuffer->lineBuffer.at(yind).length()); ++xind) {
        if (mpBuffer->lineBuffer.at(yind).at(xind) == ' ' || mpHost->mDoubleClickIgnore.c
            break;
        }
    }
    mDragSelectionEnd.setX(xind - 1);
    mPB.setX(xind - 1);
}
```

Consider this stacktrace:

```
#5  0x00007ffff6490ba3 in () at /lib/x86_64-linux-gnu/libc.so.6
#6  0x00007ffff648fff0 in qt_assert_x(char const*, int, void*)
#7  0x00005555556432b9 in QString::at(int) const
#8  0x0000555555ccf799 in TTextEdit::expandSelectionToWords()
    yind = 619
    xind = 0
#9  0x0000555555ccffdb in TTextEdit::mousePressEvent()
```

```

isOutOfBounds = true
eventPos = {xp = 23, yp = 721}
lineIndex = 619
tCharIndex = 0
cursorLocation = {xp = 0, yp = 619}
#10 0x00007ffff75af4ee in QWidget::event(QE\

```

Please suggest a fix to address the crash. The line that triggers the crash in particular is this one:

```

if (mpBuffer->lineBuffer.at(yind).at(xind) ==
QChar::Space

```

The stacktrace indicates that your program is crashing due to an assertion failure within `QString::at(int) const`, which is called as part of the `expandSelectionToWords()` function in your `TTextEdit` class. This crash is happening because the code attempts to access a character in a `QString` with an index that is out of bounds. This situation usually occurs when the index specified in the `at()` method is either negative or greater than or equal to the length of the string.

Given the code and stacktrace provided, the issue likely arises from not properly handling cases where `yind` (the line index) or `xind` (the character index within the line) refers to positions outside the valid range of `mpBuffer->lineBuffer`. Here are steps to fix this issue:

- 1. Check `yind` Against the Range of `lineBuffer`:** Before accessing `lineBuffer.at(yind)`, ensure that `yind` is within the valid range of indices for `lineBuffer`. This means checking that `yind` is greater than or equal to 0 and less than the size of `lineBuffer`.
- 2. Check `xind` Within the Loop Conditions:** When accessing characters within a line using `xind`, ensure that `xind` is also within the valid range. This means checking that `xind` is greater than or equal to 0 and less than the length of the string at `lineBuffer.at(yind)`.

Here's a modified version of your function incorporating these checks:

cpp

```
void TTextEdit::expandSelectionToWords()
{
    int yind = mPA.y();
    int xind = mPA.x();

    // Check if yind is within the valid range of lineBuffer
    if (yind >= 0 && yind < static_cast<int>(mpBuffer->lineBuffer.size()))
        for (; xind >= 0; --xind) {
            // Ensure xind is within the valid range for the current line
            if (xind < static_cast<int>(mpBuffer->lineBuffer.at(yind).size()))
                QChar currentChar = mpBuffer->lineBuffer.at(yind).at(xind);
                if (currentChar == QChar::Space
                    || mpHost->mDoubleClickIgnore.contains(currentChar)) {
                    break;
                }
            } else {
                break; // xind is out of bounds, break the loop
            }
        }
    mDragStart.setX(xind + 1);
    mPA.setX(xind + 1);

    yind = mPB.y();
    xind = mPB.x();

    // Repeat the check for yind and xind for the second part
    if (yind >= 0 && yind < static_cast<int>(mpBuffer->lineBuffer.size()))
        for (; xind < static_cast<int>(mpBuffer->lineBuffer.at(yind).size()); ++xind)
            QChar currentChar = mpBuffer->lineBuffer.at(yind).at(xind);
            if (currentChar == QChar::Space
                || mpHost->mDoubleClickIgnore.contains(currentChar)) {
                break;
            }
        }
    mDragSelectionEnd.setX(xind - 1);
```

```
mPB.setX(xind - 1);  
}
```

This code introduces checks to ensure that both `yind` and `xind` are within valid ranges before accessing elements of `mpBuffer->lineBuffer` with them. This approach should prevent the crash by avoiding out-of-bounds access to the line buffer.